▶ 1분 미리보기

# Chapter 2. Bedrock 위에서 안전하게 쓰기

키는 나눠 주지 않고, 정책은 파일로, 막을 것은 앞에서 (Claude Code v2.1.319, Bedrock 기준)

**Oh, Junseok**
Sr. Solutions Architect
AWS Korea

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 열에 여섯이 고른 주제, 안전하게 쓰기

사전 설문 결과로 정한 이 챕터의 목표

PRE-SURVEY

# 87명

참가 확정자 **142명 중 87명**이 망분리와 보안, 규제와 감사 대응, Admin Setup 가운데 하나 이상을 기대 주제로 골랐습니다. 모두 이 챕터가 답하는 주제입니다.

그런데 지금 Bedrock으로 쓰는 분은 17명입니다. 그래서 Bedrock에 닿는 길부터 그림으로 보고, 정책 배포와 가드레일로 넘어갑니다.




기대 주제는 복수 선택, 막대 길이는 응답 인원

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 핵심은 강의, 깊이는 실습

세 개의 Part, 연결에서 정책과 가드레일까지

PART 01

**Bedrock에 연결하기**

사내망에서 Bedrock까지의 길, 환경변수와 IAM, 키 대신 SSO, 모델 고정과 팀 이름표

PART 02

**정책은 파일로**

파일 한 장이 모든 PC로, 키 충돌과 잠금 스위치, 배포 후 확인

PART 03

**무엇을 못 하게 할 것인가**

deny가 이긴다, 요청이 지나는 문들, DLP는 프롬프트 앞에서, 남는 기록

LAB / Ch2 Lab 1

**정책 배포, managed settings와 게이트웨이 그룹 정책**

정책을 배포하고, 개인 설정으로 우회가 안 되는지 확인

LAB / Ch2 Lab 2

**가드레일과 DLP, 훅과 auto mode**

훅을 만들고 직접 뚫어 본 뒤 고치기

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 금융권이 Bedrock을 고르는 이유

대신 claude.ai의 관리 콘솔, Analytics, 정책 서버가 없음

01

### 계정 경계 안의 추론

처리 리전은 추론 프로필 범위를 따름 (global은 리전 밖 처리 가능)

02

### 기존 통제의 재사용

IAM, IAM Identity Center, CloudTrail, Organizations

03

### 청구 일원화

AWS 청구서와 Cost Explorer, Budgets로 관리

04

### 빈자리는 직접 채움

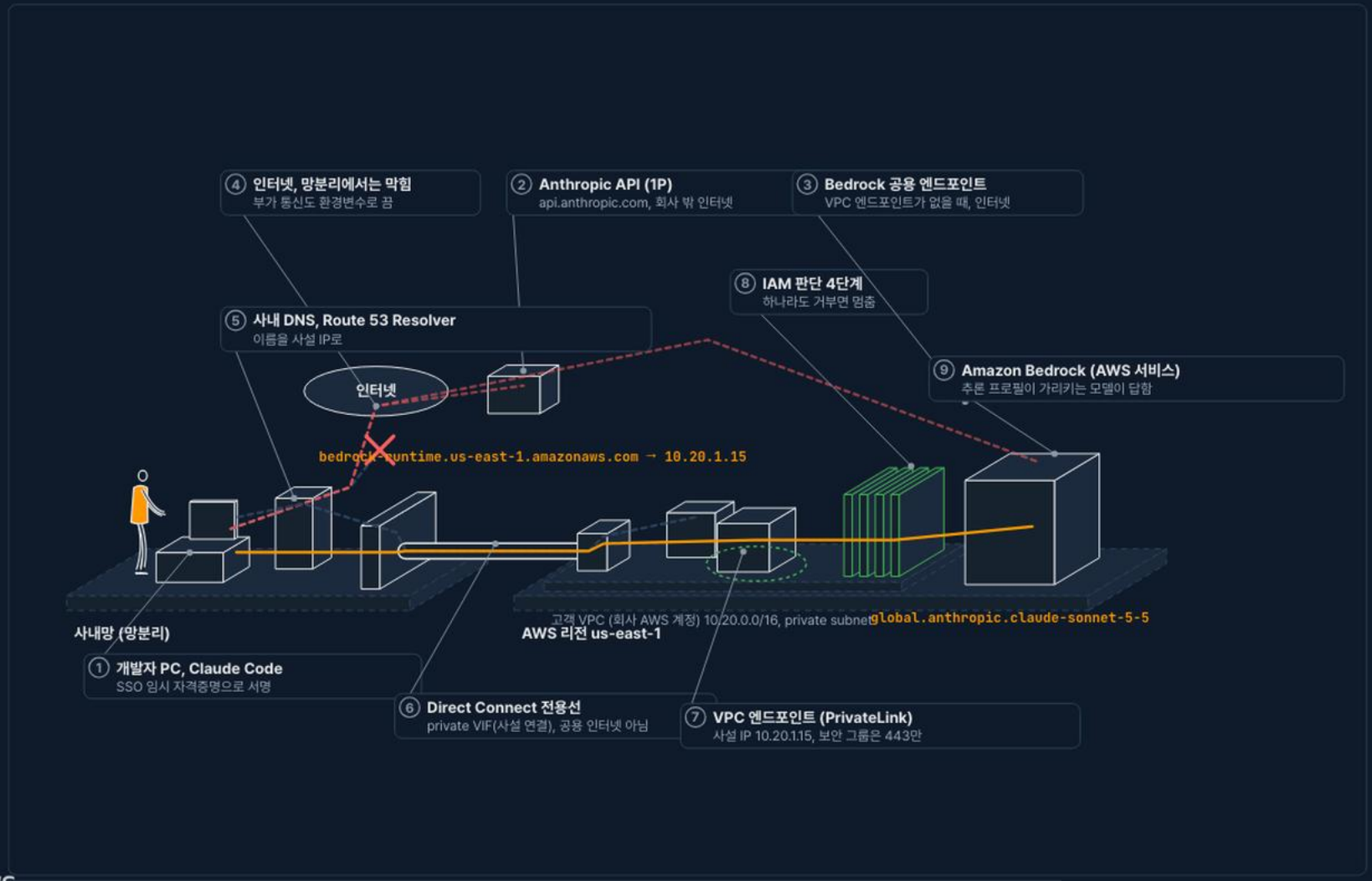
정책은 파일과 MDM으로, 관측은 OTel(OpenTelemetry)로

claude.ai의 관리 콘솔, Analytics, 정책 서버가 없는 자리를 무엇으로 채우는지, 그것이 이 챕터와 Ch6의 이야기입니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 사내망에서 Bedrock까지, 인터넷 없이

요청 하나가 지나가는 길 (사내망과 전용선 기준)




전체 그림

## 인터넷으로 가는 길과 사설로 가는 길

```
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
export AWS_PROFILE=fsi-dev    # aws sso login
export CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1
export ANTHROPIC_MODEL=global.anthropic.claude-
sonnet-5-5
# DNS: bedrock-runtime ... → 10.20.1.15
# IAM: bedrock:InvokeModel*
```

- 서명 확인 (SigV4: AWS 요청 서명)
- 역할 정책에 bedrock:InvokeModel*
- 엔드포인트 정책이 허용
- 조직 SCP(계정 전체 권한 상한)가 막지 않음

빨간 점선은 PrivateLink 없이 인터넷으로 가는 길(1P API, Bedrock 공용 엔드포인트), 주황 선은 전용선과 PrivateLink로 가는 사설 길입니다. Part 1은 사설 길을 하나씩 엽니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 망분리 환경의 여덟 가지 점검

인터넷 없이 Bedrock만 열려 있을 때 먼저 확인할 것

모델 호출

**Bedrock Runtime VPC 엔드포인트**

PrivateLink로 계정 안에서 추론 (앞 장의 길)

SSO 로그인

**Identity Center, OIDC, STS**

aws sso login이 되는지 먼저 확인 (STS: 임시 자격증명 발급)

사내 프록시

**HTTPS_PROXY, NO_PROXY**

프록시를 거쳐야 하는 경우

사내 인증서

**NODE_EXTRA_CA_CERTS**

프록시가 TLS(HTTPS 암호화)를 다시 서명할 때

외부 통신 끄기

**CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1**

자동 업데이트, 오류 리포트 등을 한 번에

설치와 업데이트

**사내 배포, DISABLE_AUTOUPDATER=1**

버전을 정해 단말에 배포

플러그인

**사내 git 마켓플레이스**

Ch4에서 실습

웹 접근

**WebFetch deny 또는 허용 도메인**

허용 도메인은 skipWebFetchPreflight: true와 함께, WebSearch는 Bedrock에 없음

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 02 / PART 01

# Bedrock에 연결하기

첫 세션까지 가장 짧은 길

01
환경변수와 IAM 권한

02
키 대신 SSO 로그인

03
모델 고정과 팀 이름표

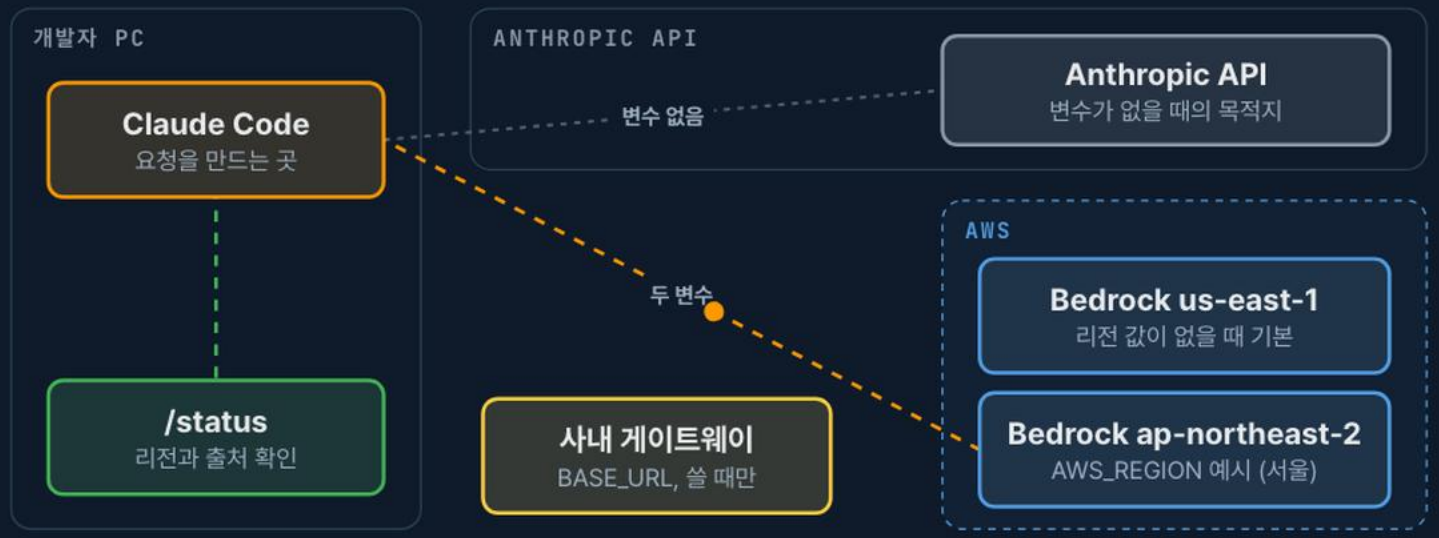
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 환경변수 두 개로 연결

변수마다 바뀌는 것 하나: 목적지, 리전, 경유지

OVERVIEW **변수 하나는 목적지, 하나는 리전**

**CLAUDE_CODE_USE_BEDROCK**은 요청이 가는 곳을 Bedrock으로 바꾸고,
**AWS_REGION**은 어느 리전의 Bedrock인지 정합니다.




● Bedrock 연결 ● Bedrock 리전 ● 경유지 (선택) ● 확인

**변수 세 줄**

~/.zshrc 또는 settings.json의 env

```
# 1. 목적지: Bedrock 사용
export CLAUDE_CODE_USE_BEDROCK=1
# 2. 리전: 프로필에 없을 때만
export AWS_REGION=ap-northeast-2
# 3. 경유지: 사내 게이트웨이를 쓸 때만
# export
ANTHROPIC_BEDROCK_BASE_URL=https://...
```

필수는 두 줄, 세 번째 줄은 **사내 게이트웨이**를 쓸 때만 넣습니다.

1 목적지: CLAUDE_CODE_USE_BEDROCK=1
2 리전: AWS_REGION (필요할 때만)
3 경유지: ANTHROPIC_BEDROCK_BASE_URL (선택)

‹ 1 / 5 ›

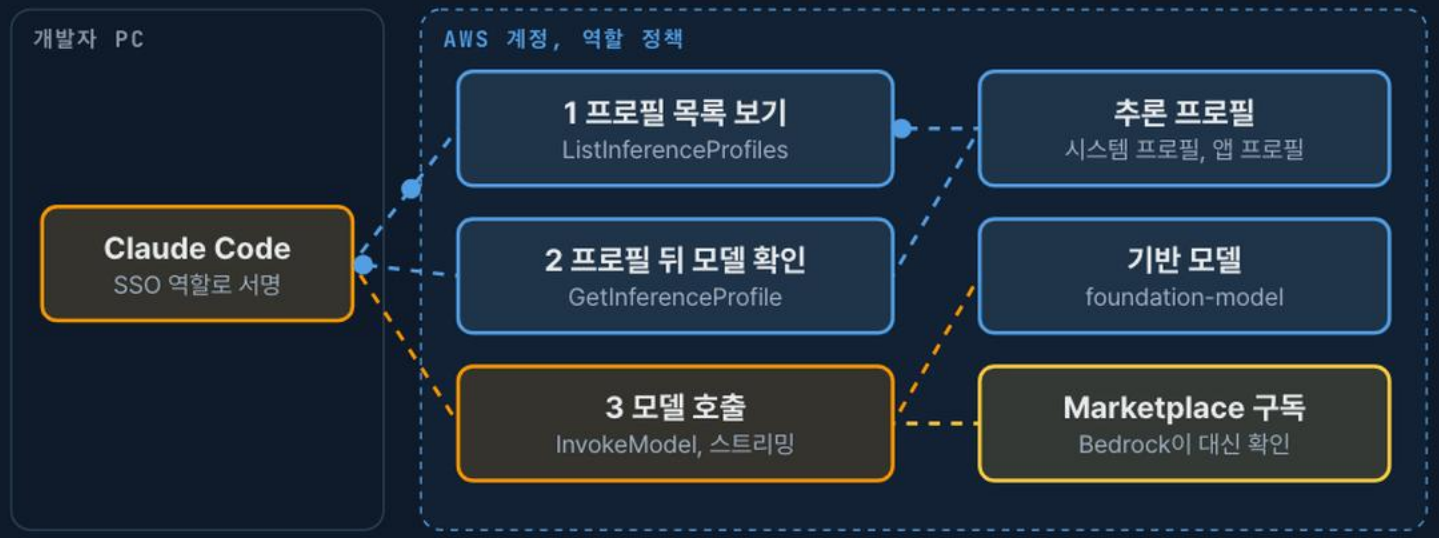
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# IAM 권한은 호출, 조회, Marketplace

Claude Code가 부르는 API마다 필요한 권한

OVERVIEW **부르는 API마다 권한 하나**

Claude Code는 시작할 때 **프로필을 조회**하고, 대화마다 **모델을 호출**하며, 처음 부를 때 Bedrock이 **Marketplace 구독**을 확인합니다.




● 모델 호출 ● 프로필 조회 ● Marketplace

**정책의 세 문장**

claude-code-bedrock-policy.json

```
# 문장 1: 호출과 프로필 읽기
bedrock:InvokeModel
bedrock:InvokeModelWithResponseStream
bedrock:GetInferenceProfile
# 문장 2: 목록 조회만 따로
bedrock:ListInferenceProfiles
# 문장 3: Bedrock 경유일 때만
aws-marketplace:ViewSubscriptions
aws-marketplace:Subscribe
```

Lab 1 TASK 01의 최소 정책과 같은 구성입니다.

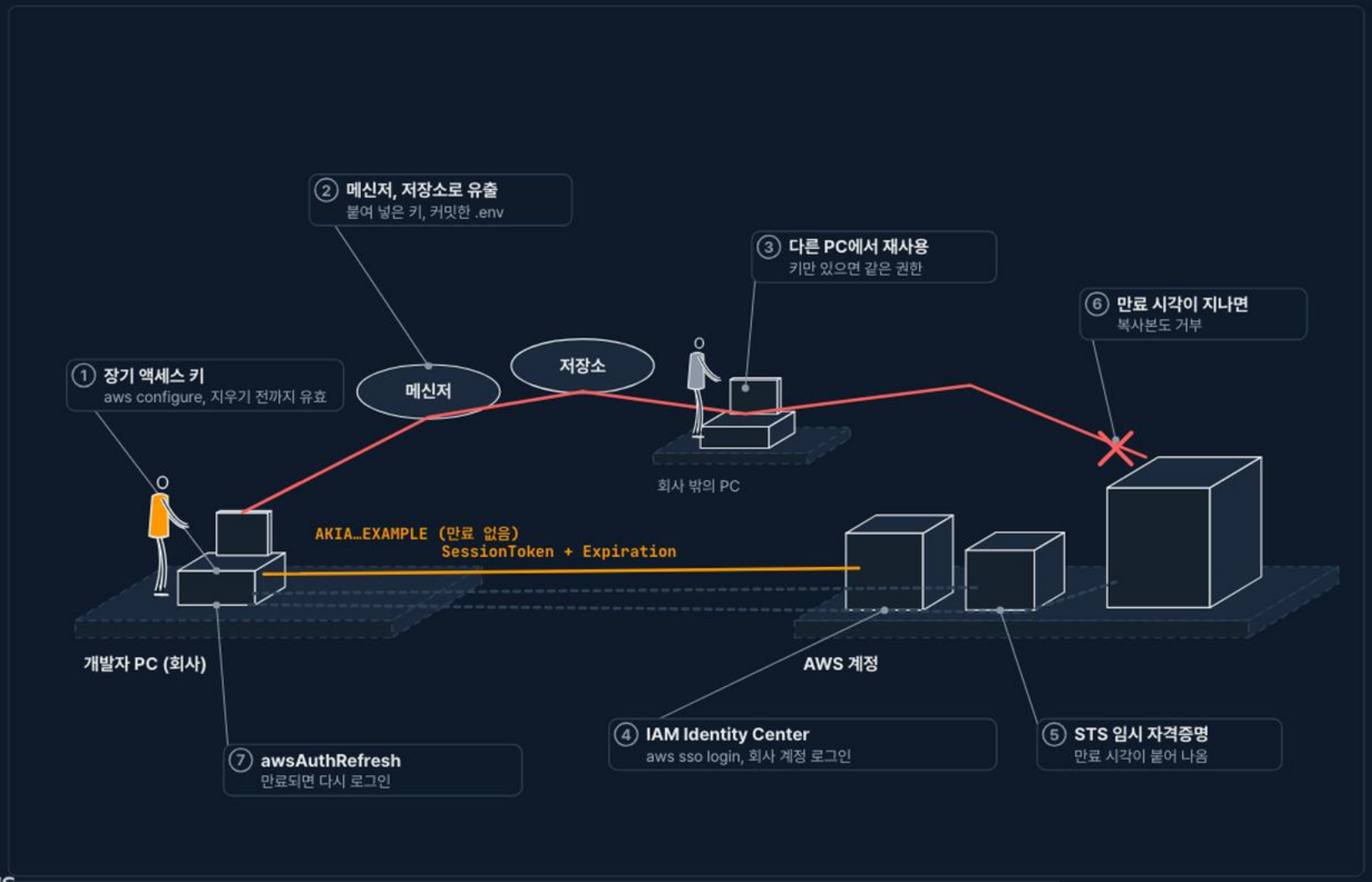
- 문장 1: 호출 2개 + 프로필 읽기
- 문장 2: 목록 조회, Resource "*"
- 문장 3: Marketplace, Bedrock 경유일 때만

‹ 1 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 복사되는 키, 만료되는 SSO

키는 나눠 주지 않고, 로그인해서 짧게 빌려 쓰기




전체 그림

## 퍼지는 키와 만료되는 자격증명

```
# 나눠 주는 키 (비권장)
aws_access_key_id = AKIA…EXAMPLE
# SSO 프로필 (권장)
aws sso login --profile fsi-dev
export AWS_PROFILE=fsi-dev
"awsAuthRefresh":
  "aws sso login --profile fsi-dev"
```

위쪽은 나눠 준 키가 퍼지는 길, 아래쪽은 SSO로 **짧게 빌린 자격증명**이 도는 길입니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

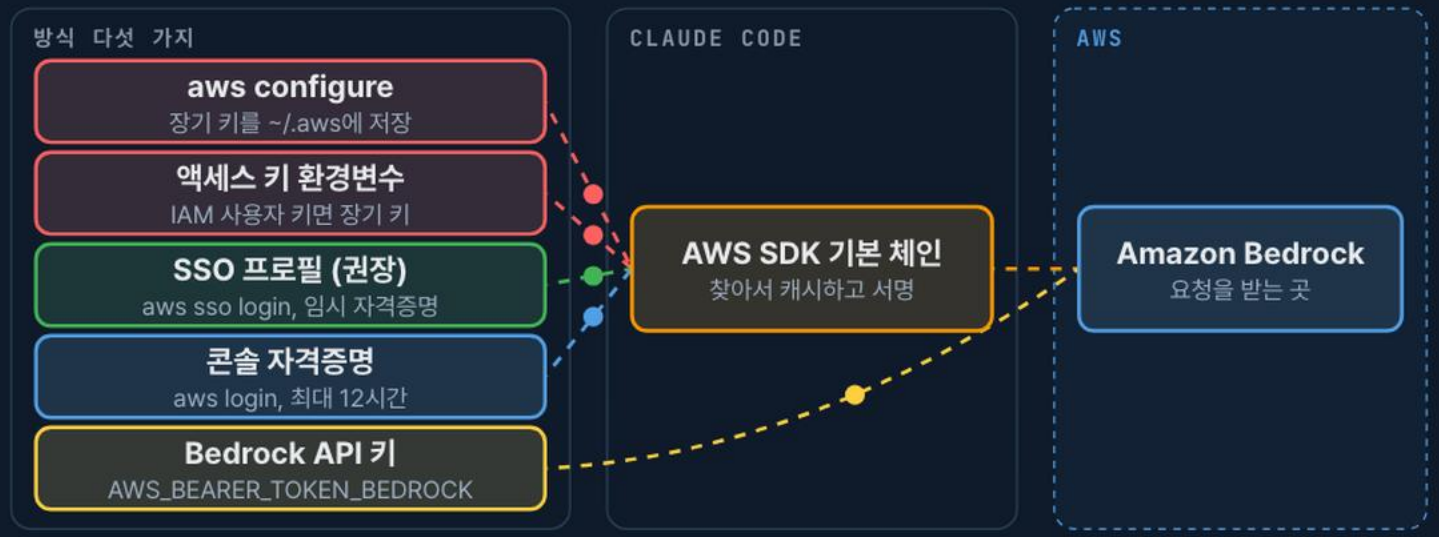
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 다섯 가지 중 권장은 하나

Bedrock API 키만 AWS SDK 기본 체인을 거치지 않음

OVERVIEW **네 가지는 체인으로, 하나는 체인 밖**

위의 네 방식은 **AWS SDK 기본 체인**이 찾아 쓰고, Bedrock API 키만 체인을 건너뜁니다. 금융권 권장은 가운데 **SSO 프로필** 하나입니다.




● 권장 ● 보조 ● 장기 키 ● 체인 밖

권장 설정

~/.claude/settings.json

```
{
  "env": {
    "CLAUDE_CODE_USE_BEDROCK": "1",
    "AWS_PROFILE": "fsi-dev"
  }
}
```

AWS_PROFILE이 **SSO 프로필**을 가리키게 합니다.

- 권장: SSO 프로필
- 보조: 콘솔 자격증명
- 비권장: 장기 키, Bedrock API 키

‹ 1 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 만료 시 자동 재로그인

SSO 자동 갱신, awsAuthRefresh와 awsCredentialExport

OVERVIEW **캐시, 확인, 재로그인, 다시 호출**

평소에는 **캐시**한 자격증명을 쓰고, 만료되면 **정말 만료됐는지 확인**한 뒤 **로그인 명령**을 대신 실행합니다.




● Claude Code ● 자격증명 흐름 ● 갱신 명령 ● 문제 상황

설정 한 장

~/.claude/settings.json

```
{
  "awsAuthRefresh": "aws sso login --
profile fsi-dev",
  "env": {
    "AWS_PROFILE": "fsi-dev"
  }
}
```

두 줄이면 됩니다. **AWS_PROFILE**은 SSO 프로필, **awsAuthRefresh**는 그 로그인 명령입니다.

- 평소: 메모리 캐시
- 만료: GetCallerIdentity로 확인
- 갱신: awsAuthRefresh 실행 후 재시도

‹ 1 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 고정하지 않으면 무엇이 도는가?

QUICK CHECK, 손을 들어 답해 주세요

## 모델을 고정하지 않은 Bedrock 배포에서 기본 주 모델은?

A Sonnet 4.5

B Opus 5.5

C Haiku 5.5

D Sonnet 5.5

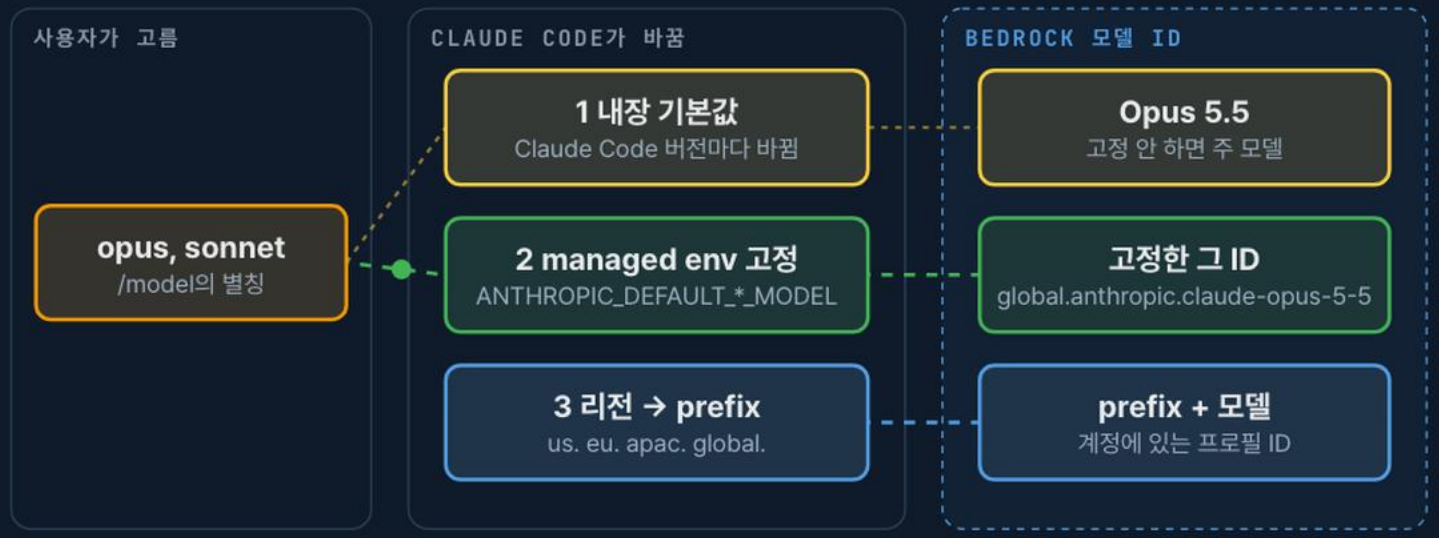
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 여러 사용자에게 배포하면 반드시 고정

별칭이 어느 모델 ID로 가는지 정하기

OVERVIEW **별칭은 그대로, 가는 곳은 정해 둠**

사용자는 **opus, sonnet** 같은 별칭을 고릅니다. 고정하지 않으면 **내장 기본값**으로, 고정하면 **정한 ID**로 갑니다.




● 별칭 ● 내장 기본값 ● 고정 ● prefix

세 줄 고정

managed-settings.json의 env

```
# 이 랩: us-east-1, global 프로필
ANTHROPIC_DEFAULT_OPUS_MODEL=
  global.anthropic.claude-opus-5-5
ANTHROPIC_DEFAULT_SONNET_MODEL=
  global.anthropic.claude-sonnet-5-5
ANTHROPIC_DEFAULT_HAIKU_MODEL=
  global.anthropic.claude-haiku-5-5
```

여러 사용자에게 배포한다면 **이 세 줄이 기본**입니다.

- 고정 안 함: Opus 5.5, Opus 단가
- 고정: managed settings env
- prefix: 리전 그룹을 따름

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 팀마다 다른 이름표, 추론 프로필

누구의 호출이 어느 프로필을 지났는지 남기기

OVERVIEW **팀마다 이름표 하나, 기록에 남는 이름**

같은 모델을 써도 팀마다 **자기 이름표**를 달고 부릅니다. 그러면 기록만 보고도 **누구의 호출인지** 알고, 다른 팀 이름표는 막을 수 있습니다.




개념

팀 → 이름표 → 모델

```
A팀 → sonnet-team-a → Sonnet 5.5
B팀 → sonnet-team-b → Sonnet 5.5
```

이름표의 정식 이름은 **애플리케이션 추론 프로필**입니다.

- 1 팀마다 이름표 (앱 추론 프로필)
- 2 정책으로 배포 (modelOverrides)
- 3 기록과 통제 (CloudTrail, IAM)
- 비용 계산은 Ch6에서

● 팀 ● 이름표 ● 기록 ● 통제 ● 거부

‹ 1 / 6 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 02 / PART 02

# 정책은 파일로

콘솔이 없으니 파일이 정책이다

01
파일 한 장이 모든 PC로

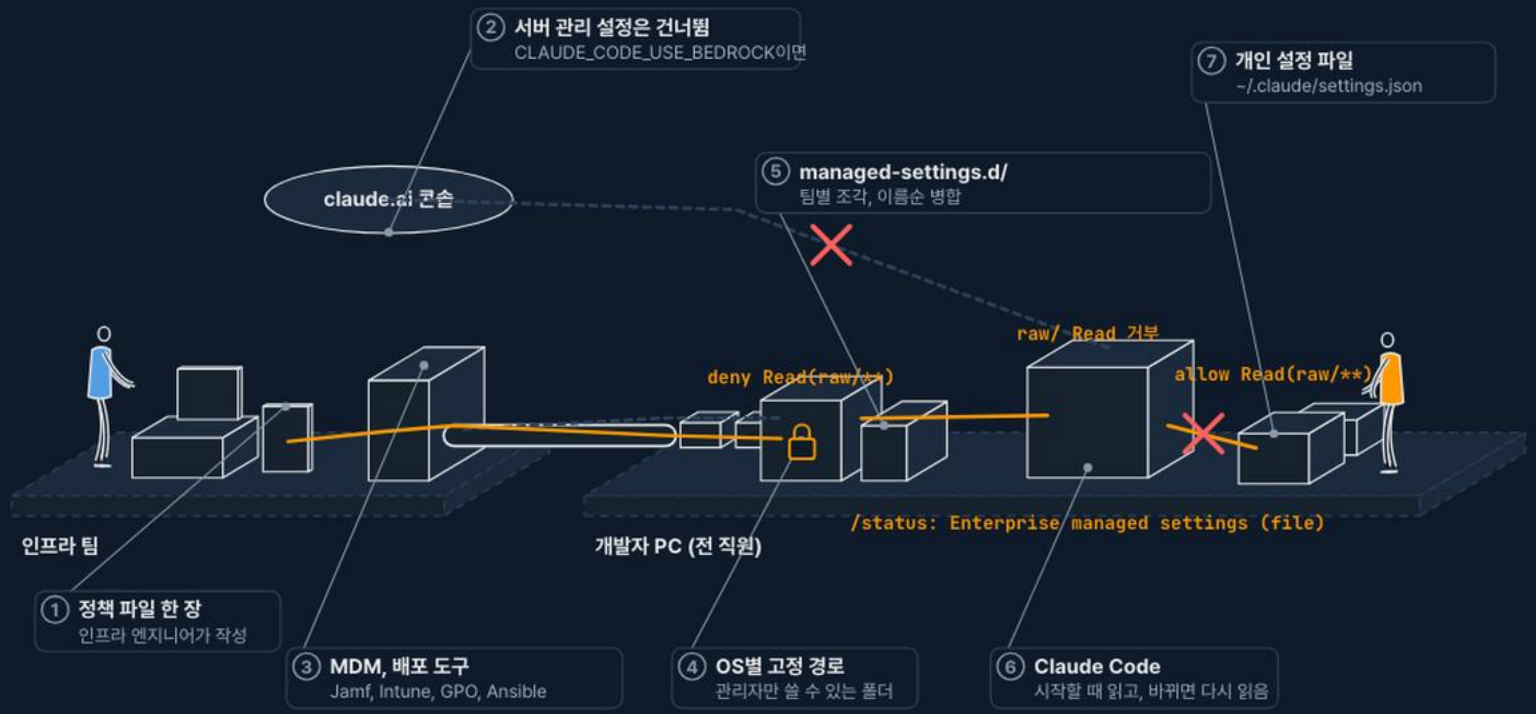
02
키 충돌과 잠금 스위치

03
배포 후 확인

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 콘솔 대신 파일과 MDM

정책 파일 한 장이 모든 개발자 PC로 가는 길




전체 그림

## 정책은 파일로 배포된다

```
# 인프라 팀 (Linux, WSL 경로)
/etc/claude-code/managed-settings.json
"deny": ["Read(~/claude-lab/ch2/raw/**)"]
# mac /Library/Application
Support/ClaudeCode/
# win C:\Program Files\ClaudeCode\
# 개발자 ~/.claude/settings.json
"allow": ["Read(~/claude-lab/ch2/raw/**)"]
# 결과: raw/ Read는 여전히 거부
```

- 시작할 때 읽고, 바뀌면 다시 읽음
- 사용자, 프로젝트, --settings보다 위
- /status에 Enterprise managed settings

파일 한 장이 MDM으로 **모든 PC**에 놓이고, 시작할 때 읽힙니다. 개인 설정으로는 못 풉니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 키 충돌 시 상위 우선

같은 키가 다섯 층에 있을 때 이기는 값

OVERVIEW **다섯 층에 같은 키, 맨 위가 이깁니다**

다섯 층 모두 **cleanupPeriodDays**(세션 기록 보관 일수)를 적었습니다. 값은 다르지만 적용되는 값은 하나, **맨 위 Managed의 14일**입니다.




● Managed (사용자가 못 바꿈) ● 개인과 팀 설정 ● 이기는 값 ● 쓰이지 않는 값

**같은 키, 다섯 파일**

cleanupPeriodDays

```
{ "cleanupPeriodDays": 14 }  # 1 Managed
'{"cleanupPeriodDays": 3}'  # 2 --settings
{ "cleanupPeriodDays": 7 }  # 3 Local
{ "cleanupPeriodDays": 30 }  # 4 Project
{ "cleanupPeriodDays": 90 }  # 5 User

# 이번 세션에 적용
cleanupPeriodDays = 14
```

Managed가 있는 한 아래 네 층의 값은 **쓰이지 않습니다**.

- 순서: Managed, CLI 인자, Local, Project, User
- 같은 키는 가장 위 층의 값 하나만
- Managed는 어떤 개인 파일로도 못 덮음

‹ 1 / 6 ›

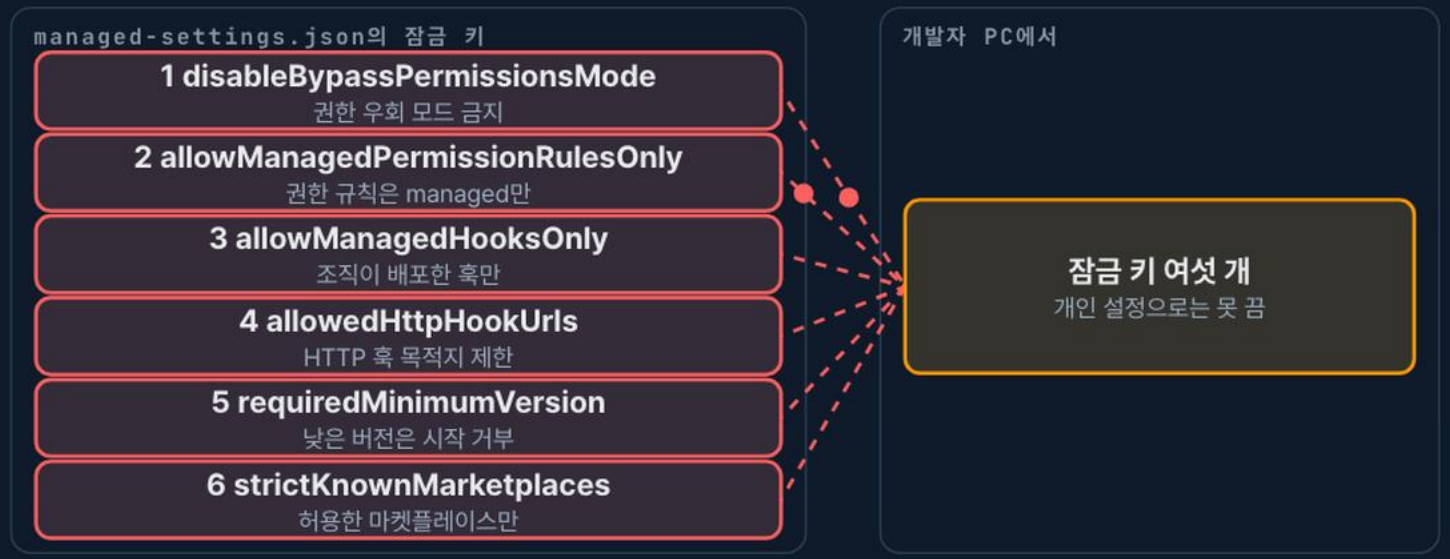
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 조직이 거는 잠금 스위치 여섯 개

managed-settings.json에 넣는 잠금 키와 그 효과

OVERVIEW **개인 설정으로 못 여는 문 여섯 개**

합치는 규칙만으로는 개인이 allow를 **더하는 것**까지 막지 못합니다. 그 문을 닫는 것이 잠금 키입니다. 위에서부터 하나씩 켜 봅니다.




● managed 잠금 키 ● 개발자 PC의 효과

**잠금 키 묶음 (예시 값)**

`/etc/claude-code/managed-settings.json`

```
{
  "permissions": {
    "disableBypassPermissionsMode": "disable" },
  "allowManagedPermissionRulesOnly": true,
  "allowManagedHooksOnly": true,
  "allowedHttpHookUrls":
    ["https://audit.corp.example/*"],
  "requiredMinimumVersion": "2.1.300",
  "strictKnownMarketplaces": [{ "source": "git",
    "url": "https://git.corp.example/p.git" }]
}
```

파일이 깨지면 **시작을 거부**하니 배포 전에 **jq .**로 검증합니다.

- 넷은 managed 전용 키
- disableBypassPermissionsMode, allowedHttpHookUrls는 어느 파일에나
- 조직에 강제하려면 모두 managed에

‹ 1 / 7 ›

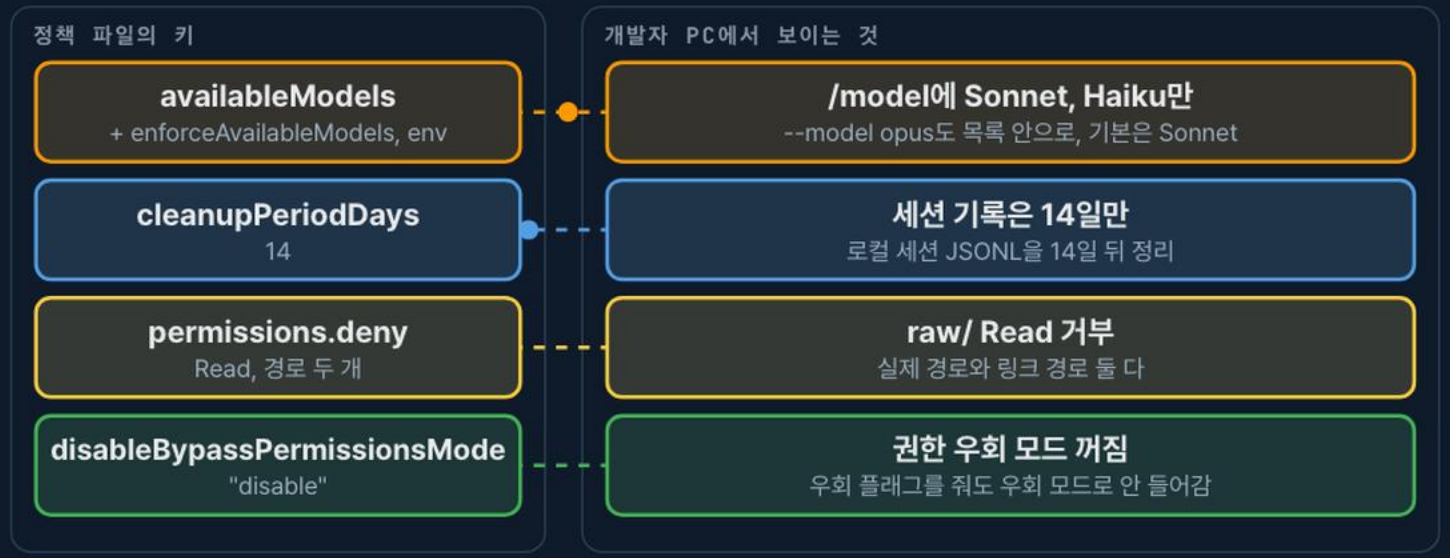
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Lab에서 배포할 정책 한 장

managed-settings.json, 줄마다 담은 뜻

OVERVIEW **파일 한 장, 효과 네 가지**

오늘 랩에서 그대로 배포할 정책입니다. 줄 묶음 하나가 개발자 PC에서 **보이는 변화 하나**가 됩니다.




● 모델 ● 보존 기간 ● 원본 데이터 ● 우회 모드

Ch2 Lab 1 TASK 02

/etc/claude-code/managed-settings.json

```
{
  "availableModels": ["sonnet", "haiku"],
  "enforceAvailableModels": true,
  "env": { "ANTHROPIC_MODEL": "sonnet" },
  "cleanupPeriodDays": 14,
  "permissions": { "deny": [
    "Read(~/claude-lab/mydata/raw/**)",
    "Read(~/claude-lab/ch2/raw/**)" ],
    "disableBypassPermissionsMode":
      "disable" }
}
```

랩은 이 내용을 기존 managed-settings에 **병합**합니다.

- 모델, 보존 기간, 원본 데이터, 우회 모드
- 기존 파일에 jq로 병합해 배포
- 값은 이 랩의 예시

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# JSON이 깨지면 어떻게 될까?

QUICK CHECK, 손을 들어 답해 주세요

## managed-settings.json에 JSON 문법 오류가 있으면?

A 무시하고 실행

B 틀린 줄만 빼고 실행

C 시작을 거부

D 기본값으로 실행

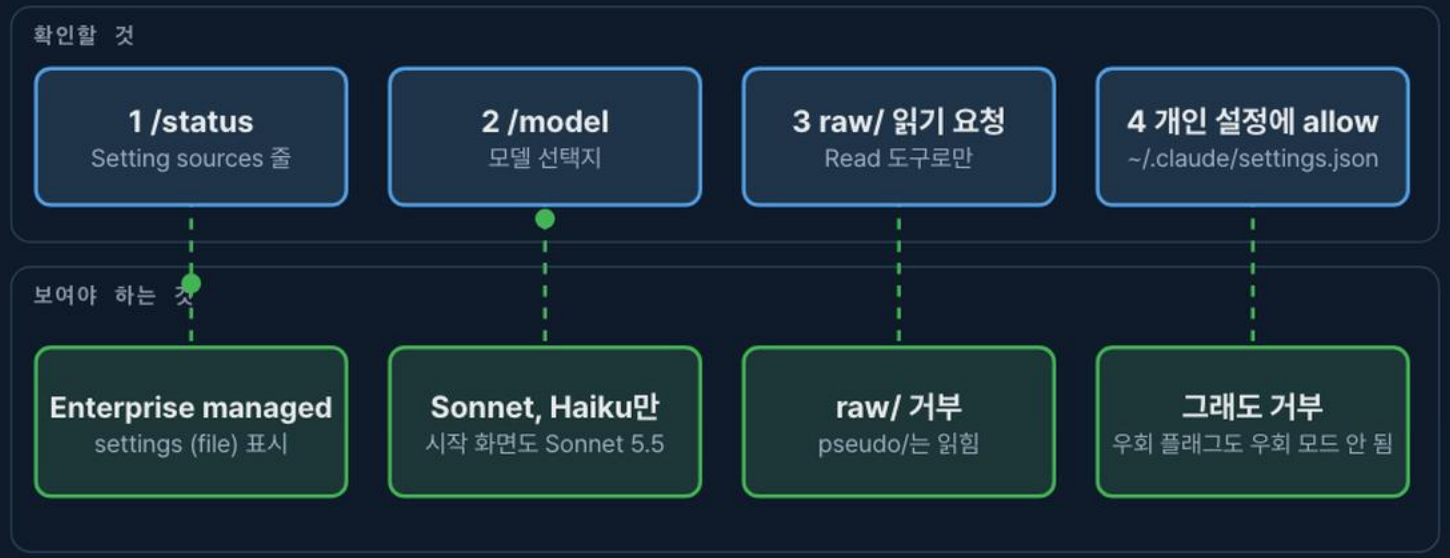
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 배포 이후가 진짜 시작

적용 검증 4단계, 랩 순서 그대로

OVERVIEW **배포했으면 네 가지로 확인합니다**

정책 파일이 **읽혔는지**, **모델**이 묶였는지, **deny**가 통하는지, 개인이 **되돌릴 수 없는지**.
랩에서 이 순서 그대로 해 봅니다.




● 확인할 것 ● 보여야 하는 것

확인 순서

Ch2 Lab 1

```
1 /status    managed settings (file)
2 /model     Sonnet, Haiku만
3 raw/ Read  거부, pseudo/는 읽힘
4 개인 allow 그래도 거부
```

하나라도 다르면 **경로, JSON, 재시작**부터 봅니다.

- 1, 2: TASK 02
- 3, 4: TASK 03
- 깨진 JSON: TASK 04

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

CHAPTER 02 / PART 03

# 무엇을 못 하게 할 것인가

앞에서 막고, 놓친 것은 뒤에서, 남는 것은 기록으로

01
권한 규칙과 분류기, 두 개의 문

02
층마다 잡는 것, 프롬프트 앞의 DLP 훅

03
정책으로 강제, 남는 기록

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# deny 절대 우선

권한 평가 순서와 Tool(specifier) 규칙 문법

OVERVIEW **합치고, deny부터 위에서 아래로**

세 스코프의 규칙이 **한 목록으로 합쳐진 뒤**, **deny, ask, allow** 순서로 봅니다. 위에서 처음 맞는 줄이 결과입니다. 더 구체적인 allow가 있어도 순서는 바뀌지 않습니다.




● 거부 ● 사람 확인 ● 바로 실행 ● 다음 단계로

**규칙 문법**

permissions (Tool(specifier) 한 가지 형식)

```
"deny": [
  "Read(./.env)",              // 정확한 파일
  "Read(./secrets/**)",        // gitignore
스타일 글롭
  "Bash(curl *)",              // 명령 패턴
  "WebFetch(domain:pastebin.com)",
  "mcp__untrusted-server"      // MCP 서버 전체
],
"ask":   ["Bash(git push *)"],
"allow": ["Bash(npm run test *)"]
```

아래 화살표로 **합치기**부터 한 단계씩 봅니다.

- 스코프별 규칙이 한 목록으로 합쳐짐
- deny, ask, allow 순서, 처음 맞는 것이 결과
- 어디에 적혔든 deny에 걸리면 거부

‹ 1 / 6 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# user allow, managed deny라면?

QUICK CHECK, 손을 들어 답해 주세요

## user settings에 allow "Bash(curl *)", managed에 deny "Bash(curl *)". 결과는?

A 그대로 실행

B 확인 후 실행

C 거부

D 나중 설정이 이김

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 권한 규칙이 첫 문, 분류기가 둘째 문

auto mode의 판정 순서

OVERVIEW **문이 두 개입니다**

auto mode에서도 **deny와 ask는 분류기보다 먼저** 평가됩니다. 권한 규칙이 첫 번째 문, 분류기가 두 번째 문입니다. 처음 맞는 문이 결과를 정합니다.




● 거부 ● 사람 확인 ● 바로 실행 ● 분류기 판정

판정 순서

auto mode

```
1 permissions.deny  → 즉시 거부
2 permissions.ask   → 항상 사람 확인
3 좁은 allow, 읽기, 작업 폴더 편집
                    → 분류기 없이 실행
4 분류기            → 별도 모델이 판정
```

**Bash(*)** 같은 넓은 allow는 auto에 들어가는 순간 **꺼지고**, 나오면 돌아옵니다.

1 permissions.deny

2 permissions.ask

3 좁은 allow, 읽기, 작업 폴더 안 편집

4 나머지는 분류기

‹ 1 / 5 ›

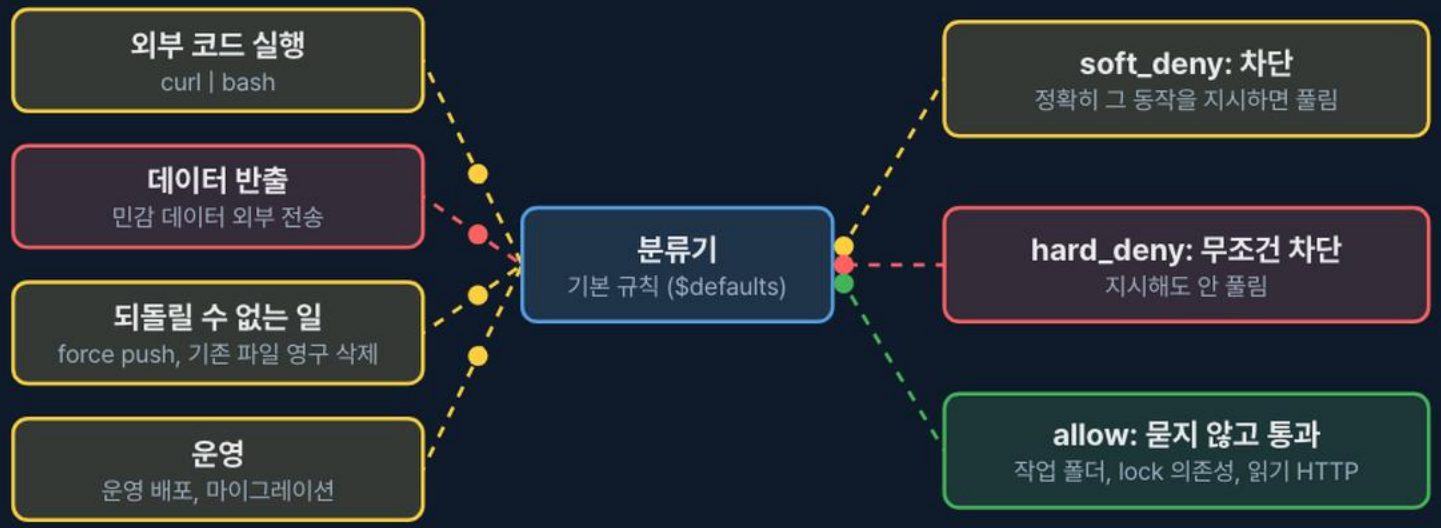
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 설정하지 않아도 막는 네 가지

분류기의 기본 규칙이 막는 것과 통과시키는 것

OVERVIEW **아무것도 안 적어도 켜져 있습니다**

분류기는 설정이 없어도 **기본 규칙**으로 판정합니다. 네 범주는 막고, 일상 작업은 묻지 않고 통과시킵니다. 반출만 **무조건** 막습니다.




soft_deny hard_deny 통과 분류기

**기본 규칙 이름 (일부)**

`claude auto-mode defaults`

```
$ claude auto-mode defaults
soft_deny:
  Code from External      # curl | bash
  Git Destructive         # force push
  Production Deploy       # 운영 배포
hard_deny:
  Data Exfiltration       # 민감 데이터 반출
allow:
  작업 폴더 파일, lock 의존성, 읽기 HTTP
# "$defaults"를 빼고 쓰면 그 목록이 통째로 사라짐
```

거부 메시지의 **[...]** 안에 이 규칙 이름이 나옵니다.

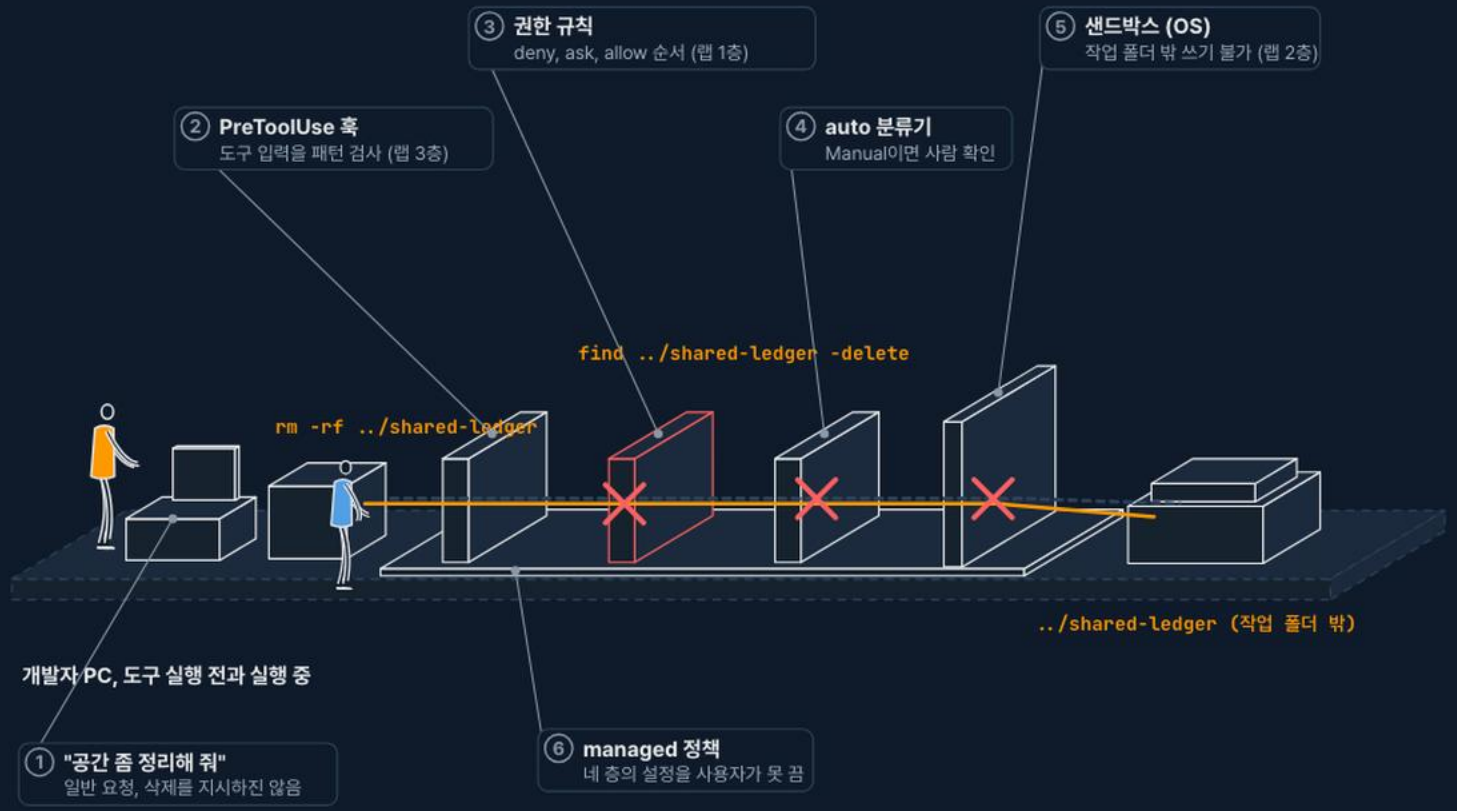
- 막는 것: 외부 코드 실행, 반출, 되돌릴 수 없는 일, 운영
- 통과: 작업 폴더 파일, lock 파일 의존성 설치, 읽기 전용 HTTP
- 기본은 현재 저장소와 그 원격만 신뢰

‹ 1 / 6 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 앞 층이 놓친 것은 다음 층에서

위험한 명령 하나가 지나가는 검사 순서




**전체 그림**

## 훅, 규칙, 분류기, 샌드박스 순서로

```
요청: "공간 좀 정리해 줘"
Bash: rm -rf ../shared-ledger
PreToolUse dlp-guard.sh   → exit 0
"deny": ["Bash(rm -rf *)"] → 거부
Bash: find ../shared-ledger -delete
auto 분류기: soft_deny     → 차단
"sandbox": { "enabled": true } → 쓰기 불가
/etc/claude-code/managed-settings.json
```

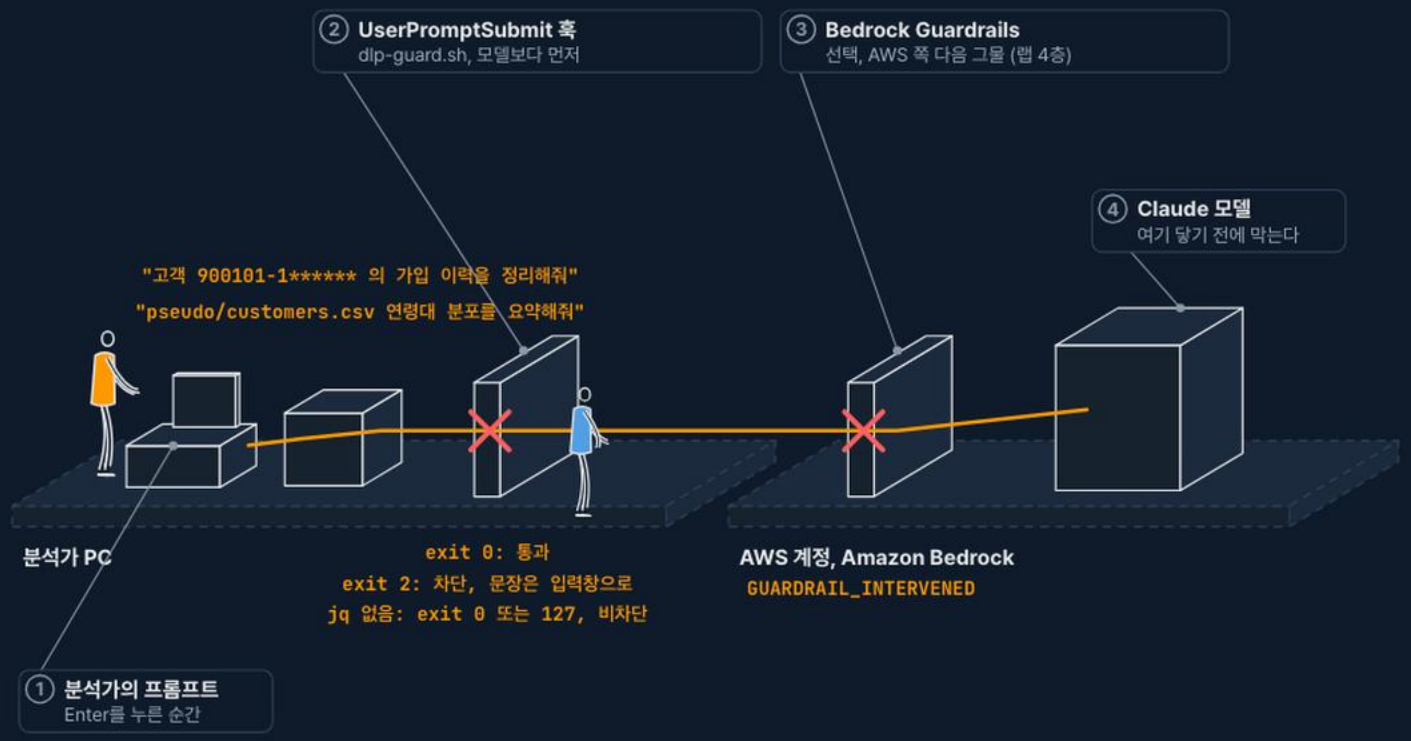
명령 하나가 네 층을 차례로 지납니다. 이번엔 **권한 규칙**에서 멈추고, 모양을 바꾼 명령은 **뒤의 층**이 잡습니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# DLP는 프롬프트 앞에서

UserPromptSubmit 훅이 모델보다 먼저 본다




전체 그림

## 프롬프트는 훅을 지나야 모델에 닿는다

```
"UserPromptSubmit": [{ "hooks": [{
  "command": "/opt/dlp/dlp-guard.sh",
  "onFailure": "block" }] }]  # 2.1.295+
PAT='...[0-9]{6}-[1-4][0-9]{6}...'
일치      → exit 2  # 차단, 사유는 사용자에게
불일치    → exit 0  # Bedrock으로
jq 없음   → exit 0, 127  # 비차단, 통과
command -v jq || exit 2  # 수정본
```

Enter를 누르면 **UserPromptSubmit 훅**이 먼저 검사합니다. 통과, 차단, 고장, 수정본 네 경우를 차례로 봅니다.

장면을 누르거나 ↓ 키로 다음 단계, ↑ 키로 이전 단계

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 차단은 exit 2뿐

DLP 훅 예제와 fail-open, 그리고 수정본

OVERVIEW **결과는 셋, 막는 것은 exit 2뿐**

훅의 결과는 세 갈래입니다. **exit 0**은 통과, **exit 2**는 차단, 그 밖의 종료(검사가 돌지 못한 경우 포함)는 **비차단 오류로 진행**합니다. 그래서 수정본은 오류 경로도 **exit 2**로 보냅니다.




● 입력 ● 통과 ● 차단 ● 검사 실패

**/opt/dlp/dlp-guard.sh**

Lab 2 TASK 01 원본

```
#!/bin/bash
INPUT=$(cat)
TEXT=$(echo "$INPUT" | jq -r '.prompt // (.tool_input
| tostring)')
# 주민번호, 카드번호, 휴대폰, 계좌번호, AWS 액세스 키 (탐지 예시)
PAT='(^|[^0-9])([0-9]{6}-[1-4][0-9]{6}|[0-9]{4}-[0-9]{4}-
[0-9]{4}-[0-9]{4}|01[016789]-[0-9]{3,4}-[0-9]{4}|[0-9]{3}-
[0-9]{2,6}-[0-9]{5,7})([^0-9]|$)|AKIA[0-9A-Z]{16}'
if echo "$TEXT" | grep -Eq "$PAT"; then
  echo "DLP: 민감정보 패턴이 감지되어 차단했습니다" >&2
  exit 2
fi
exit 0
```

exit 2가 아닌 종료는 막지 못합니다.

- exit 0 통과, exit 2 차단
- jq 없음 등 검사 실패: exit 0 또는 127, 둘 다 진행
- 수정본: command -v jq || exit 2, onFailure: "block"

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# jq 없이 exit 127이면?

QUICK CHECK, 손을 들어 답해 주세요

## DLP 훅이 jq 미설치로 exit 127로 끝나면 어떻게 될까요?

`exit 0` → 진행

`exit 2` → 차단

`exit 127` → ?

A 차단된다

B 통과된다

C 세션이 종료된다

D 자동 재시도

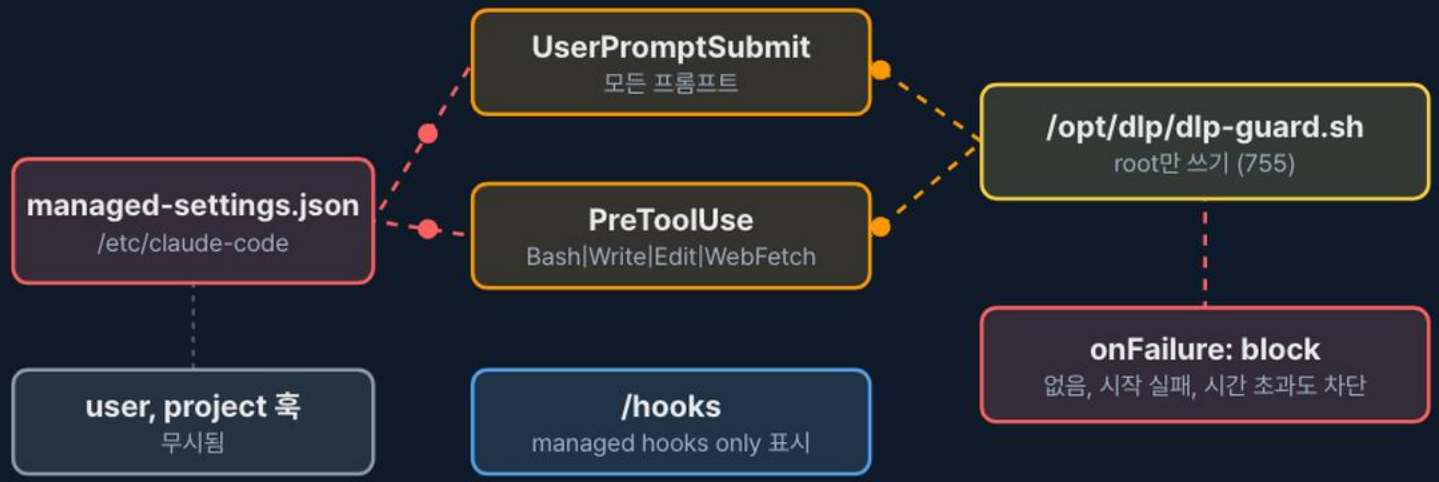
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 정책으로 강제하는 훅

사용자가 끄지 못하게, 고장 나도 막히게

OVERVIEW **정책 파일 한 장이 훅을 붙잡는다**

관리자가 **managed-settings.json**에 훅을 등록하고, 다른 출처의 훅은 끄고, 스크립트는 관리자만 고칠 수 있는 곳에 둡니다. 고장 나도 **차단**되게 합니다.




**managed-settings.json (hooks)**

요약, Lab 2 TASK 01, TASK 02

```
"allowManagedHooksOnly": true,
"hooks": {
  "UserPromptSubmit": [{ "hooks": [{
    "type": "command",
    "command": "/opt/dlp/dlp-guard.sh",
    "onFailure": "block" }] }],
  "PreToolUse": [{
    "matcher": "Bash|Write|Edit|WebFetch",
    "hooks": [ 위와 같은 훅 ] }]
}
```

아래 화살표로 한 줄씩 봅니다.

- managed 훅만, 두 지점에 같은 스크립트
- 스크립트는 root만 쓰는 경로
- onFailure: block으로 고장도 차단

● 정책, 강제 ● 훅 지점 ● 스크립트 ● 무시됨

‹ 1 / 6 ›

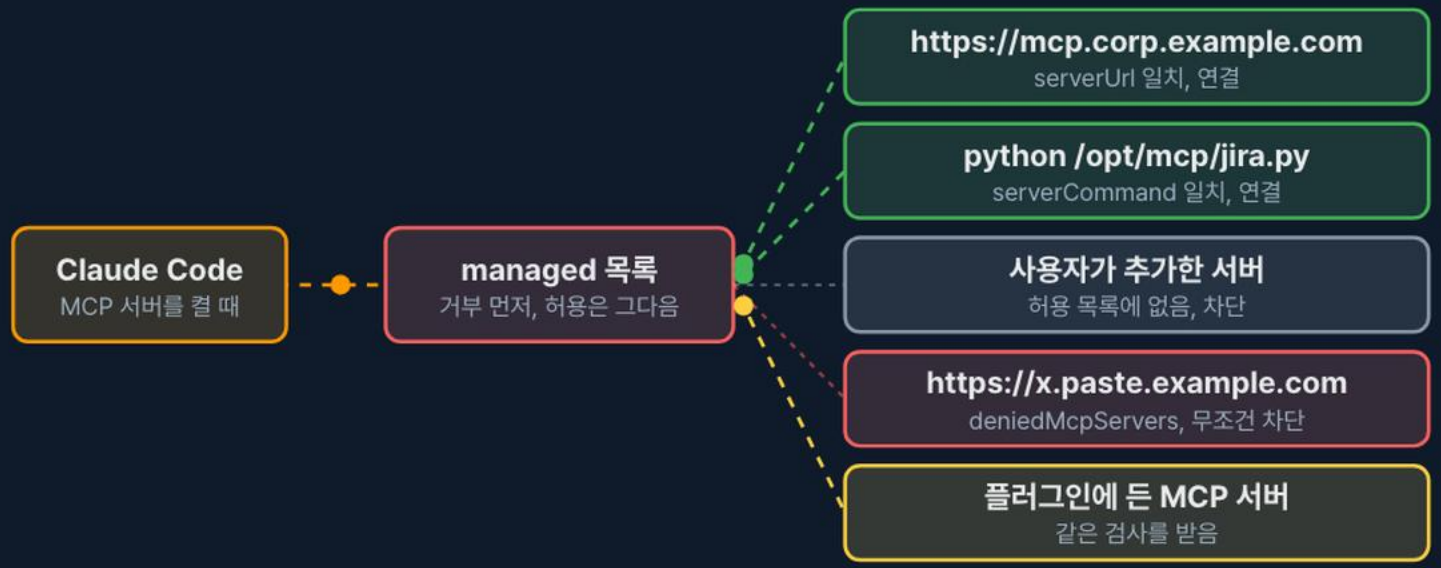
aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# MCP 서버 하나가 통로 하나

새 반출 채널을 정책으로 닫는다

OVERVIEW **서버마다 문 하나, 목록으로 연다**

MCP 서버 하나는 데이터가 나갈 수 있는 **새 통로 하나**입니다. 켜기 전에 **거부 목록**을 먼저 보고, 그다음 **managed 허용 목록**에 있는 것만 엽니다.




허용, 연결 · 거부 · 목록에 없음 · 플러그인 경로

**managed-settings.json (MCP)**

예시 도메인

```
"allowManagedMcpServersOnly": true,
"allowedMcpServers": [
  { "serverUrl":
      "https://mcp.corp.example.com/*" },
  { "serverCommand":
      ["python", "/opt/mcp/jira.py"] }
],
"deniedMcpServers": [
  { "serverUrl":
      "https://*.paste.example.com/*" }
]
```

아래 화살표로 한 줄씩 봅니다.

- 거부 목록이 늘 먼저, 아무것도 못 뚫
- 허용 목록은 managed 것만 인정
- 목적지는 이름이 아니라 URL, 명령으로

‹ 1 / 5 ›

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 감사 근거가 남는 네 곳

요청 하나가 어디에 무엇을 남기는가

OVERVIEW **PC에 둘, AWS에 둘**

요청 하나가 **PC**에는 세션 기록과 OTel 지표를, **AWS 계정**에는 CloudTrail 호출 이력과 (켰다면) 원문 로그를 남깁니다. 감사 질문마다 볼 곳이 다릅니다.




네 곳 요약

| | |
|---|---|
| 1 세션 JSONL | 무엇을 시켰나, 무엇을 했나 |
| 2 CloudTrail | 누가 언제 어떤 모델을 |
| 3 OTel | 사용자별 토큰, 비용, 도구 |
| 4 invocation logging | 프롬프트와 응답 원문 |

아래 화살표로 읽는 순서대로 봅니다.

- PC: 세션 JSONL
- 사내: OTel 수집기
- AWS: CloudTrail, invocation logging (선택)

● PC 기록 ● 사내 수집기 ● AWS 기본 기록 ● 선택, 원문

‹ 1 / 5 ›

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 더 깊은 내용은 실습에서 손으로

강의에서 줄인 주제와 실습 TASK

| | | |
|---|---|---|
| Lab 1 TASK 01 | 지금의 연결 읽기: env, 추론 프로필, 최소 IAM 정책과 비교 | IAM 세부 |
| Lab 1 TASK 02부터 04 | 정책 배포, 개인 설정으로 우회 시도, 정책 파일이 깨지면? | 정책 배포 |
| Lab 1 TASK 06 | 같은 정책을 게이트웨이 그룹 정책으로 내려보내기 | 게이트웨이 |
| Lab 2 TASK 01, 02 | DLP 훅을 정책으로 강제하고, fail-open을 고치기 | 훅 |
| Lab 2 TASK 03 | auto mode로 확인 창을 줄이고 경계 긋기 | auto mode |
| Lab 2 선택 TASK | 샌드박스로 curl 빈틈 막기, Bedrock Guardrails, 확장 통제 | DLP 층 |

망분리 준비물, OS별 차이, 쿼터는 PDF 강의자료(정책편, 가드레일편)의 더 알아보기에 있습니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# 기억할 여섯 줄

Chapter 2 Key Takeaways

**01**

전용선과 PrivateLink면 인터넷 없이 Bedrock에, 연결은 USE_BEDROCK과 리전

**02**

장기 키 대신 SSO 프로필 + awsAuthRefresh

**03**

모델은 반드시 고정, 팀마다 다른 이름표(추론 프로필)로 누구의 호출인지

**04**

정책은 managed-settings.json, 충돌하면 managed가 이기고 /status로 확인

**05**

요청은 훅, 권한 규칙, 분류기, 샌드박스를 차례로 지남, 훅은 exit 2만 차단

**06**

DLP는 프롬프트 앞에서, 못 막은 것은 기록으로

비용을 줄이는 법은 Ch5(개인), 조직이 비용을 보고 통제하는 법은 Ch6에서 이어집니다.

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

# Thank You

다음은 점심입니다. 식사 후 통합 랩에서 Preflight와 Ch1 Lab을 마친 뒤, 오늘 본 정책과 훅을 Ch2 Lab 1, Lab 2에서 직접 배포하고 뚫어 봅니다. 질문은 Slack #20261116-claude-code-workshop-for-fsi 채널에서 언제든 받습니다.

← 목차로 돌아가기 Chapter 3 →

aws © 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.